ACCESO A DATOS - 2.º DAM

UP 1 – Ficheros JSON

Introducción a las APIs

1. ¿Qué es una API?

API son las siglas de Application Programming Interface (interfaz de programación de aplicaciones). Es un conjunto de reglas y mecanismos que permite a un programa utilizar funciones o consultar datos ofrecidos por otro sistema.

Por ejemplo, una aplicación puede consultar una previsión meteorológica, comprobar el estado de un envío o mostrar un mapa sin tener que generar esos datos por sí misma.

2. ¿Por qué se utilizan?

Utilidad	Ejemplo en una aplicación
Reutilizar servicios	Mostrar un mapa utilizando los datos de un proveedor externo.
Acceder a información actualizada	Consultar horarios, resultados o información meteorológica.
Conectar sistemas	Comunicar una tienda con un servicio de pagos o transporte.
Separar responsabilidades	Una aplicación móvil muestra información que gestiona un servidor.

Una API no tiene por qué ser pública ni estar en Internet. En esta ampliación nos centraremos en las APIs web, que se consultan mediante HTTP.

3. ¿Qué ocurre cuando consultamos una API web?

Paso	Qué sucede
1. Petición	El cliente solicita un recurso al servidor mediante una dirección (URL).
2. Procesamiento	El servidor localiza o prepara la información solicitada.
3. Respuesta	El servidor devuelve un estado HTTP y, si procede, datos que la aplicación puede interpretar.

En una consulta sencilla suele utilizarse el método HTTP GET, destinado a obtener información. Una respuesta correcta puede incluir el código 200 OK y un cuerpo en formato JSON.

4. ¿Qué relación tiene con JSON?

HTTP permite intercambiar mensajes entre cliente y servidor; JSON es uno de los formatos que pueden contener esos mensajes. No son lo mismo: HTTP interviene en la comunicación y JSON representa los datos.

La ventaja para nosotros es que el JSON recibido puede tratarse con JSONObject y JSONArray, igual que el JSON que hemos leído anteriormente desde un fichero.

Un ejemplo: la API de Star Wars

SWAPI (Star Wars API) es una API pública con información organizada sobre personajes, planetas, películas, vehículos y naves del universo Star Wars.

05. Consultar un personaje

Dirección de la API: https://swapi.dev/api/people/1/

Al solicitar ese recurso se obtiene información sobre Luke Skywalker. Este es un extracto de la respuesta (se han omitido otras propiedades):

People Instance

```
GET /api/people/1/
```

```
HTTP 200 OK
Content-Type: application/json
Vary: Accept
Allow: GET, HEAD, OPTIONS
```


```
{
    "name": "Luke Skywalker",
    "height": "172",
    "mass": "77",
    "hair_color": "blond",
    "skin_color": "fair",
    "eye_color": "blue",
    "birth_year": "19BBY",
    "gender": "male",
    "homeworld": "https://swapi.dev/api/planets/1/",
    "films": [
        "https://swapi.dev/api/films/1/",
        "https://swapi.dev/api/films/2/",
        "https://swapi.dev/api/films/3/",
        "https://swapi.dev/api/films/6/"
    ],
    "species": [],
    "vehicles": [
        "https://swapi.dev/api/vehicles/14/",
        "https://swapi.dev/api/vehicles/30/"
    ],
    "starships": [
        "https://swapi.dev/api/starships/12/",
        "https://swapi.dev/api/starships/22/"
    ],
    "created": "2014-12-09T13:50:51.644000Z",
    "edited": "2014-12-20T21:17:56.891000Z",
    "url": "https://swapi.dev/api/people/1/"
}
```


El JSON contiene propiedades sencillas, como name, y un array denominado films. La propiedad homeworld incluye otra URL de la propia API: permite consultar, por separado, los datos del planeta de origen del personaje.

Elemento	Interpretación
people/1/	Recurso que identifica al personaje consultado.
name	Nombre del personaje.
height y mass	Valores enviados por la API como cadenas, no como números JSON.
films	Array de direcciones de recursos relacionados.
homeworld	Dirección para consultar otro recurso: un planeta.

06. Conectar con SWAPI desde Java

La misma consulta que hacemos desde el navegador puede realizarse desde Java. Utilizaremos HttpClient (incluido en Java 17) para enviar una petición GET a SWAPI y recuperar la respuesta como texto.

A continuación, convertiremos el texto recibido en un JSONObject para consultar los datos de Luke Skywalker. Este ejemplo une los dos pasos: conexión con la API e interpretación del JSON.

```
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import org.json.JSONObject;
import org.json.JSONArray;

public class ConsultaStarWars {

  public static void main(String[] args)
    throws IOException, InterruptedException {

      String url = "https://swapi.dev/api/people/1/";
      HttpClient cliente = HttpClient.newHttpClient();
      HttpRequest peticion = HttpRequest.newBuilder()
         .uri(URI.create(url)).GET().build();

      HttpResponse<String> respuesta = cliente.send(
         peticion, HttpResponse.BodyHandlers.ofString());

      if (respuesta.statusCode() == 200) {
         JSONObject personaje = new JSONObject(respuesta.body());
         System.out.println("Nombre: " + personaje.getString("name"));
         System.out.println("Altura: " + personaje.getString("height"));

         JSONArray peliculas = personaje.getJSONArray("films");
         System.out.println("Peliculas: " + peliculas.length());
      } else {
         System.out.println("Error HTTP: " + respuesta.statusCode());
      }
}
```

En el código, HttpRequest indica qué recurso solicitamos, cliente.send(...) envía la petición y respuesta.body() obtiene el contenido. El código de estado 200 indica que la petición ha tenido éxito. Después, JSONObject permite consultar las propiedades igual que cuando leíamos un fichero JSON. Observa que SWAPI devuelve height como texto ("172"), no como número JSON.

07. Lo que estudiaremos más adelante

Con Spring Boot profundizaremos en las peticiones y respuestas HTTP, las APIs REST y la creación de nuestros propios servicios. Por ahora es suficiente con comprender por qué existen las APIs, cómo se consulta un recurso y por qué JSON es útil para intercambiar información.